50.051 PLC – Project Report

"Retro–vision" ASCII Video Engine

Ong Xuan — 1008044
Nguyen Quoc Dung (Tristan) — 1008017

April 19, 2026

Code repository: https://github.com/TristanNguyen04/retro-vision-ascii-video-engine

Contents

1 Context and Objectives **3**
1.1 Background . . . 3
1.2 Problem Statement . . . 3
1.3 Project Scope and Success Criteria . . . 3
1.4 Reader Guide . . . 4

2 System Design and Architecture **4**
2.1 High-Level Architecture . . . 4
2.2 Module Hierarchy . . . 4
2.3 End-to-End Data Flow . . . 5
2.4 State-Machine Orchestration . . . 5
2.5 Temporal Alignment Logic . . . 6
2.6 Compression Algorithms . . . 6
2.6.1 Run-Length Encoding (RLE) . . . 6
2.6.2 Delta Encoding . . . 7
2.6.3 Huffman Coding . . . 8
2.6.4 Compressed Size Comparison . . . 12

3 Constraint Satisfaction **12**
3.1 Constraint-to-evidence matrix . . . 12
3.2 File input and output in detail . . . 14
3.3 Parser robustness and incorrect-input detection . . . 15
3.4 Constraint pipeline (diagram) . . . 16

4 Development Process and Team Collaboration **16**
4.1 Development timeline and phases . . . 17
4.2 Work split for a two-person team . . . 17
4.3 Version control with GitHub . . . 17
4.4 Integration strategy and interface contracts . . . 18
4.5 Reproducibility across team machines . . . 18

5 Challenges, Trade-offs, and Engineering Decisions **19**
5.1 Heterogeneous inputs and parser robustness . . . 19
5.2 Temporal synchronization and audio-driven highlighting . . . 19
5.3 Raw fidelity, storage footprint, and compression modes . . . 20
5.4 State-machine orchestration and maintainability . . . 20

5.5 Risk handling, testing, and debugging 21

6 Outcomes, Learning, and Future Work 21

6.1 Project outcomes 21
6.2 User experience: compression and preview are core 21
6.3 What we learned 22
6.4 Future improvements 22
6.5 Closing remarks 22

1. Context and Objectives

1.1. Background

In modern software systems, multimedia processing is often implemented through mature but opaque third-party libraries. While these tools are efficient, they can hide the low-level mechanisms that connect binary file structures, timing control, and output rendering. This project was designed to study those mechanisms directly by implementing a complete multimedia-to-text pipeline in pure C.

The system takes visual and audio inputs and converts them into adaptive ASCII animation. Instead of relying on external frameworks for parsing, synchronization, and rendering, the project implements these capabilities from first principles. This approach supports the course objective of practicing clean C programming, explicit memory and file management, and deterministic system behavior.

1.2. Problem Statement

This project bridges raw binary data and human-readable text output. The primary challenge is to build a robust C-based engine that manually parses heterogeneous inputs (BMP frame files, WAV audio data, and JSON configuration), then synchronizes them through finite-state-machine logic without external libraries.

The central technical problem is temporal alignment: each visual frame must be mapped to the correct audio sample window so that amplitude-driven highlighting is coherent over time. In parallel, the system must remain resilient to malformed inputs through strict parser validation and explicit error handling.

1.3. Project Scope and Success Criteria

The implemented scope is a complete file-processing workflow:

1) parse configuration, image frames, and audio input files;
2) orchestrate frame-by-frame processing with a state-machine-based engine;
3) generate raw ASCII render output and process logs;
4) produce compressed output and support preview playback for practical usage.

Project success is evaluated through four criteria:

- **correctness**: frame processing and audio-reactive behavior execute without undefined transitions;
- **robustness**: parsers detect invalid or corrupted inputs and fail safely;
- **compliance**: implementation aligns with course constraints (clean C, parser design, file I/O, state machines, minimal dependencies);
- **usability**: compressed output plus preview reduce storage and improve user workflow compared with raw text-only output.

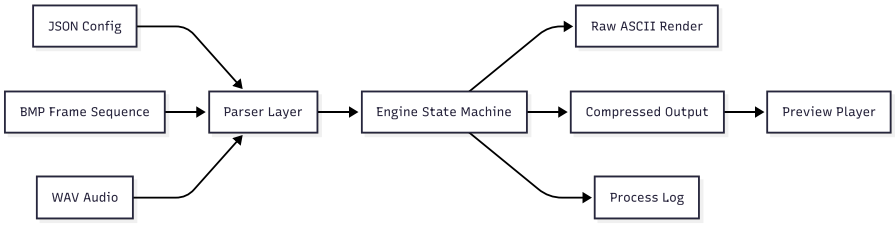


Figure 1: High-level project pipeline used as a reading guide for the report.

1.4. Reader Guide

The rest of this report is organized as follows. Section 2 explains the architecture and data flow in detail, including module hierarchy and state-machine orchestration. Section 3 maps the implementation directly to the project constraints and grading criteria. Section 4 describes the development process for a two-member team and the integration strategy across subsystems. Section 5 discusses technical challenges and trade-offs encountered during implementation. Section 6 concludes with outcomes, lessons learned, and future improvements.

2. System Design and Architecture

2.1. High-Level Architecture

The system is designed as a two-layer architecture that separates content generation from content delivery.

- **Layer A — generation pipeline:** custom parsers load configuration and media inputs, while the engine orchestrates synchronization and frame-by-frame ASCII rendering.
- **Layer B — delivery pipeline:** rendered frames are compressed into a compact file format, then replayed through a preview path that reconstructs and displays frames.

This separation improves maintainability and directly supports user-facing needs: high-fidelity raw output for inspection and compressed output for practical storage and playback.

2.2. Module Hierarchy

The repository is organized around clear functional boundaries:

- **Parser modules** (`src/parsers`): JSON/config parsing, WAV parsing, and BMP parsing.
- **Engine and rendering modules** (`src/components`): sequence path logic, ASCII conversion, rendering, compressed render I/O, and engine orchestration.
- **Compression modules** (`src/compressions`): compression/decompression dispatch and algorithm implementations (`none`, `rle`, `delta`, `huffman`).
- **Public interfaces** (`include/...`): typed APIs and shared data structures per subsystem.
- **Validation modules** (`tests/...`): subsystem-level tests for parser, component, and compression behavior.

The resulting hierarchy keeps file-format concerns, orchestration logic, and storage optimization logic decoupled while still integrating through explicit interfaces.

2.3. End-to-End Data Flow

At runtime, the engine consumes three inputs:

1) JSON configuration (frame range, frame naming rules, palette, threshold, compression settings);
2) WAV audio stream;
3) BMP frame sequence.

The engine first validates and prepares context (configuration, audio metadata, frame range), then processes each frame by loading image data, mapping it to the corresponding audio interval, rendering ASCII, and writing outputs. The system produces:

- raw ASCII render text;
- compressed frame output;
- process log for traceability and debugging.

The preview program then reads the compressed output file, reconstructs compression context (including Huffman FSM metadata when applicable), decompresses frame payloads, and prints animated ASCII to the terminal.

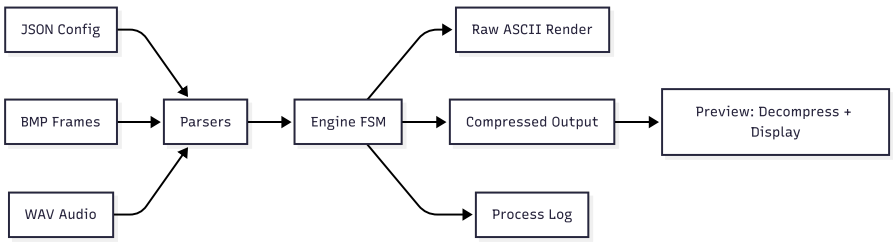


Figure 2: End-to-end architecture from heterogeneous inputs to user-facing outputs.

2.4. State-Machine Orchestration

The core orchestrator is implemented as a deterministic finite state machine in the engine. The major states are: `LOAD_CONFIG`, `LOAD_WAV`, `PREPARE_SEQUENCE`, `COLLECT_STAT`, `PREPARE_COMPRESSION`, `OPEN_OUTPUTS`, `PROCESS_FRAME`, and `WRITE_SUMMARY`, with terminal states `DONE` and `ERROR`.

Two design choices are important:

- **Pre-processing for compression readiness:** the `COLLECT_STAT` and `PREPARE_COMPRESSION` stages prepare codec-specific context (notably Huffman symbol statistics and FSM tables) before frame emission.
- **Fail-fast transitions:** any stage-level failure transitions directly to `ERROR`, providing clear control flow and predictable cleanup behavior.

This FSM-centric structure improves readability, isolates stage responsibilities, and supports robust error propagation across heterogeneous processing steps.

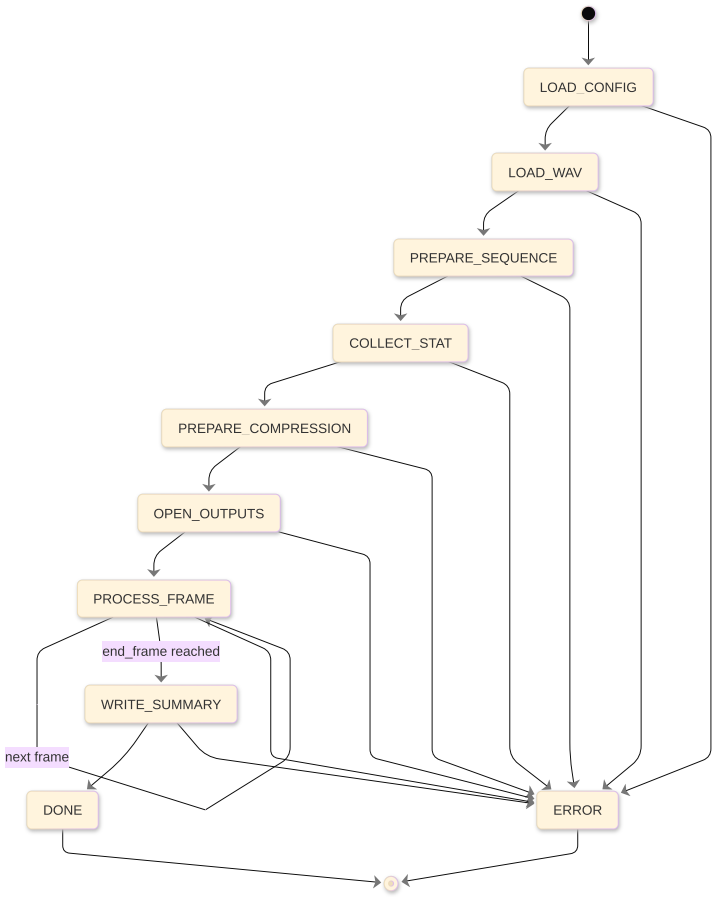


Figure 3: State-machine orchestration used by the engine runtime.

2.5. Temporal Alignment Logic

Temporal alignment is handled by mapping each frame's relative index to an audio sample range using configured frames-per-second and WAV sample rate. For frame index i, the engine computes:

$$\texttt{start_sample} = \frac{i \cdot \texttt{sample_rate}}{\texttt{fps}}, \qquad \texttt{end_sample} = \frac{(i+1) \cdot \texttt{sample_rate}}{\texttt{fps}}. \tag{1}$$

The average audio amplitude over this interval drives a highlight decision against a configured threshold. This mechanism ensures that visual emphasis in the ASCII output is synchronized with local audio intensity, producing coherent audio-reactive animation rather than independent frame and sound streams.

2.6. Compression Algorithms

To reduce the storage size of the ASCII text files generated by the renderer, several compression algorithms are investigated and implemented. These algorithms are selected to represent different compression paradigms, ranging from simple run-based encoding to probabilistic coding methods.

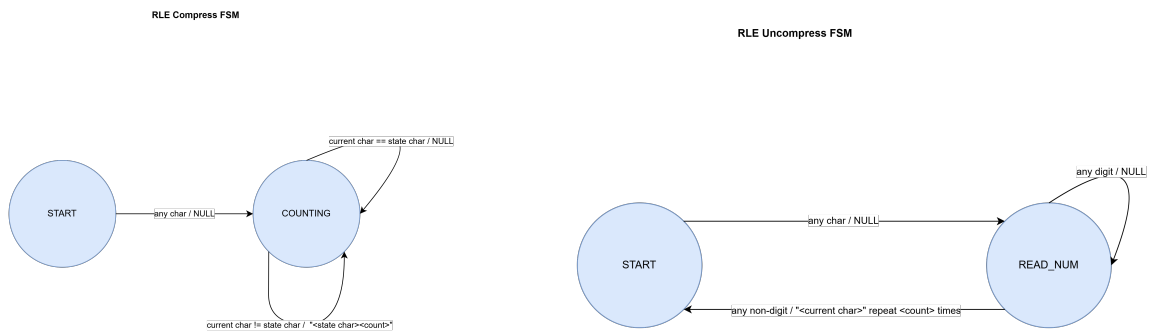
2.6.1. Run-Length Encoding (RLE)

Run-Length Encoding (RLE) is a simple lossless compression technique that reduces redundancy by encoding consecutive repeated characters as a single value and a count. Instead of storing each character individually, RLE represents a sequence as a pair consisting of the character and the number of its repetitions.

Example

$$AAAAABBBCC \xrightarrow{\text{RLE encode}} A5B3C2$$

FSMs



(a) RLE Compress FSM (b) RLE Decompress FSM

Figure 4: RLE Compression and Decompression FSMs

The Run-Length Encoding (RLE) compression and decompression processes are implemented using finite state machines (FSMs), as shown in Figures 4a and 4b.

For **compression**, the FSM has two states: *Start* and *Counting*. The first character initializes a run, after which consecutive identical characters are counted. When a different character is encountered, the current run is encoded as a character–count pair and appended to the output, and a new run begins. After processing the input, the final run is written. This approach performs compression in a single pass.

For **decompression**, the FSM consists of three states: *Start*, *Read Number*, and *Write*. A character is first read and stored, followed by digits that form its run length. Once the number is complete, the character is repeated accordingly and written to the output. The FSM then returns to the Start state to process the next segment, with checks to handle invalid inputs.

2.6.2. Delta Encoding

Delta encoding is applied across consecutive ASCII frames produced by the renderer. Instead of storing each frame independently, the algorithm encodes only the differences between the current frame and the previous frame. This exploits temporal redundancy, as **consecutive frames in a video are often highly similar**. The first frame is stored in its original form, as there is no prior reference frame for comparison.

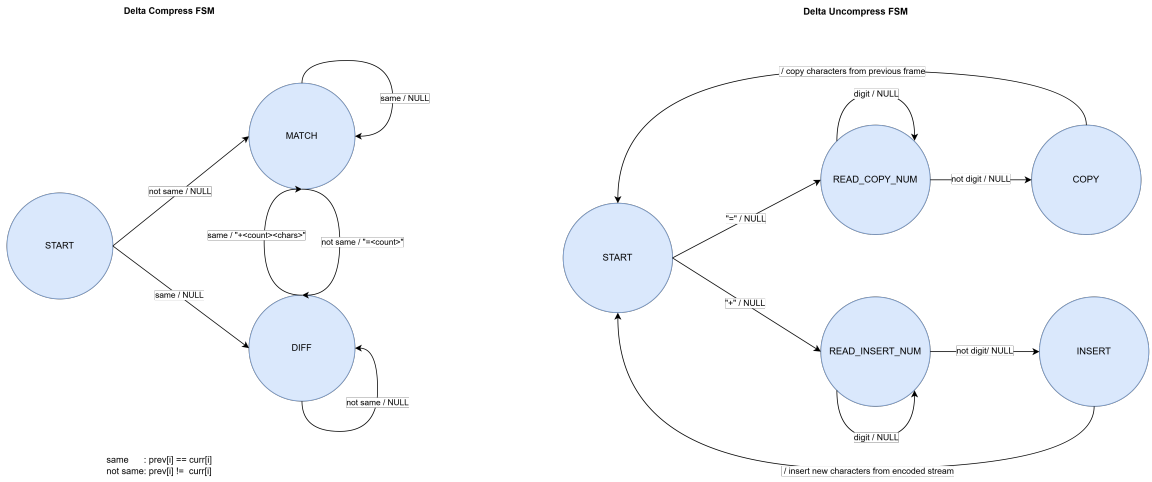
Example
Frame 1: AAAAABBBBBCCCC
Frame 2: AAAAADDDEECCCC

$$Frame_1 \xrightarrow{\text{Delta encoding}} Frame_1$$
$$Frame_2 \xrightarrow{\text{Delta encoding}} =5+5\text{DDDEE}=5$$

where:

- **=n** indicates that the next n characters are identical to those in the previous frame
- **+n[substring]** indicates that the next n characters differ from the previous frame and are replaced by the specified substring from the current frame.

FSMs



(a) Delta Compress FSM (b) Delta Decompress FSM

Figure 5: Delta Compression and Decompression FSMs

The Delta compression and decompression processes are implemented using finite state machines (FSMs), as shown in Figures 5a and 5b.

For **compression**, the FSM consists of three states: *Start*, *Match*, and *Diff*. The algorithm compares the current frame with a reference (previous) frame character by character. In the Match state, consecutive identical characters are grouped and encoded as a copy operation. In the Diff state, differing characters are grouped and encoded as an insertion. The FSM transitions between these states whenever the comparison result changes, ensuring that both matching and differing segments are efficiently encoded in a single pass.

For **decompression**, the FSM operates with states *Start*, *Read Copy Number*, *Read Insert Number*, *Copy*, and *Insert*. The encoded stream is parsed by first identifying whether the next operation is a copy or insert. The corresponding run length is then read, after which characters are either copied from the reference frame or inserted directly from the encoded input. The FSM repeats this process until the entire output is reconstructed, with checks to ensure the encoded format is valid.

2.6.3. Huffman Coding

Huffman coding is a variable-length encoding technique that assigns shorter bit sequences to more frequent symbols and longer sequences to less frequent ones. Unlike RLE and Delta encoding, which operate directly on characters, Huffman coding produces a **bit stream** to achieve compression beyond the limits of ASCII representation.

Huffman Coding without FSM

The standard approach consists of two stages: tree construction and encoding/decoding. A Huffman tree is first built using symbol frequencies, where each leaf node represents a character and the path from the root defines its binary code. Encoding is performed by replacing each character with its corresponding bit sequence, while decoding traverses the tree bit by bit until a leaf node is reached.

Example

Original text: `AAAAABBBCC`

Step 1: Compute frequency of each character.

Character	Frequency
A	5
B	3
C	2

Table 1: Huffman Frequency Table for `AAAAABBBCC`

The frequencies determine how the Huffman tree is constructed, where less frequent symbols are placed deeper in the tree.

Step 2: Build the Huffman tree.

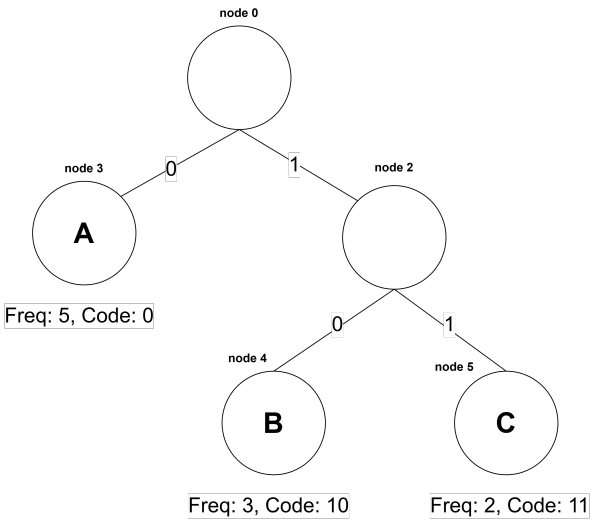


Figure 6: Huffman Tree for `AAAAABBBCC`

The Huffman tree is constructed using a greedy approach with a min-heap data structure. First, all characters with non-zero frequencies are inserted into the heap as individual leaf nodes, where each node stores a character and its frequency.

At each step, the two nodes with the smallest frequencies are extracted from the heap. These two nodes are then merged to form a new internal node, whose frequency is the sum of the two. The extracted nodes become the left and right children of this new node. The merged node is then inserted back into the heap.

This process is repeated until only one node remains in the heap, which becomes the root of the Huffman tree. The resulting tree ensures that characters with higher frequencies are closer to the root, leading to shorter binary codes.

Step 3: Encode the text using the generated codes.

$$\texttt{AAAAABBBCC} \xrightarrow{\text{Huffman coding}} 00000\ 101010\ 1111$$

Each character is replaced by its corresponding binary code obtained from the Huffman tree. This is done by traversing from the root to the leaf node of each character, appending a ‘0‘ for a left edge and a ‘1‘ for a right edge. The resulting codes are concatenated to form a continuous bit stream.

Since more frequent characters have shorter codes, the overall length of the encoded bit stream is reduced compared to fixed-length representations such as ASCII. The bit stream is stored using a bit-level buffer rather than characters, allowing efficient packing of bits into bytes.

Step 4: Decode the bit stream.
The encoded bit stream is decoded by traversing the Huffman tree from the root according to each bit. A ‘0‘ corresponds to moving to the left child, while a ‘1‘ corresponds to moving to the right child. When a leaf node is reached, the corresponding character is emitted, and traversal restarts from the root for the next sequence of bits.
For example, the bit stream

00000 101010 1111

is decoded by repeatedly following paths in the tree, reconstructing the original text:

`AAAAABBBCC`

In practice, the bit stream is read using a bit-level reader, which extracts bits from the compressed byte array. The decoding process requires processing one bit at a time, resulting in a time complexity of $O(N)$ with relatively **high constant overhead** due to frequent tree traversal.

Optimization: Huffman Decoding with FSM
To reduce the overhead of bit-by-bit tree traversal, the Huffman decoding process can be optimized using a finite state machine (FSM). Instead of processing one bit at a time, the FSM reads K bits in each step and performs a table lookup to determine the output symbols and next state.

Step 1: Convert the Huffman tree into an FSM.
Each internal node in the Huffman tree is treated as a state. For a given state, all possible 2^K bit combinations are precomputed. Each entry stores:

- the sequence of decoded characters,
- the number of bits consumed,
- the next state.

This flattens multiple levels of tree traversal into a single transition.

Step 2: Process K bits at a time.

Instead of reading one bit, the decoder reads K bits from the bit stream and uses them as an index into the FSM table. This allows multiple characters to be decoded in a single step, skipping intermediate tree traversal.

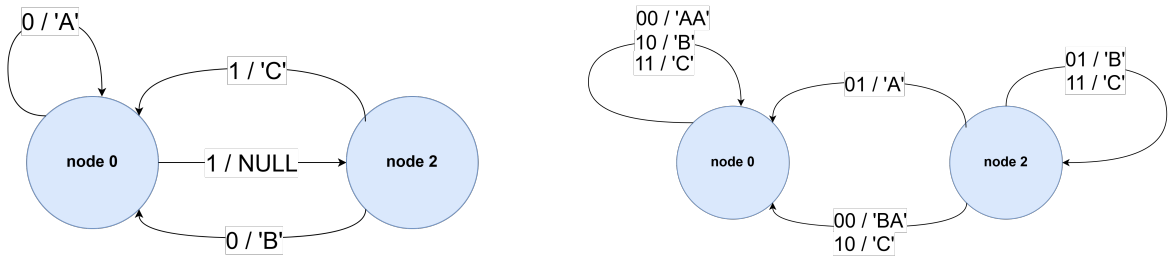
Simplified example with $K = 2$

State	Input	Output	Next State	Bits Used
S0 (Root)	00	AA	S0	2
S0 (Root)	01	A	S1	1
S0 (Root)	10	B	S0	2
S0 (Root)	11	C	S0	2
S1 (Root.right)	00	BA	S0	2
S1 (Root.right)	01	B	S1	2
S1 (Root.right)	10	C	S0	2
S1 (Root.right)	11	C	S1	2

Table 2: Huffman FSM transition table for $K = 2$ with bit consumption

Note1: Only non-terminal nodes are included as FSM states, since reaching a leaf node results in symbol emission and an immediate reset to the root, making leaf nodes transient and unnecessary as explicit states.

Note 2: Although each transition is selected using a K-bit input, the number of bits consumed may be less than K. This occurs when a symbol is fully decoded before all K bits are used, leaving the remaining bits to be processed in the next step.



(a) Original Huffman FSM ($K = 1$) (b) Optimized Huffman FSM ($K = 2$)

Figure 7: Comparison between standard and $K = 2$ Huffman decoding FSMs

Figure 7 compares standard bit-by-bit Huffman decoding with the optimized $K = 2$ FSM approach. In the original FSM, each transition processes a single bit, producing **one character at a time** and incurring higher overhead due to repeated tree traversal.

In contrast, the optimized FSM **processes $K = 2$ bits per transition**, combining multiple traversal steps into a single table lookup. This allows multiple characters to be emitted in one step, reducing the number of transitions required.

For example, consider the bit stream

$$000001010101111$$

The decoding proceeds as follows (for $K = 2$ lookahead):

$$\begin{aligned}
&\underline{00}0001010101111 \rightarrow (AA,\ 2,\ S0)\\
&\underline{00}01010101111 \rightarrow (AA,\ 2,\ S0)\\
&\underline{01}010101111 \rightarrow (A,\ 1,\ S1)\\
&\underline{10}10101111 \rightarrow (B,\ 2,\ S0)\\
&\underline{10}101111 \rightarrow (B,\ 2,\ S0)\\
&\underline{10}1111 \rightarrow (B,\ 2,\ S0)\\
&\underline{11}11 \rightarrow (C,\ 2,\ S0)\\
&\underline{11} \rightarrow (C,\ 2,\ S0)
\end{aligned}$$

Each tuple (output, bits_used, next_state) represents the result of one FSM transition.
The decoded output is:

`AAAAABBBCC`

Each transition is selected using a K-bit lookahead, but may consume fewer than K bits. For instance, "01" produces "A" while consuming only 1 bit, leaving the remaining bit to be processed in the next step. This allows the decoder to combine multiple traversal steps into a single lookup, improving efficiency compared to standard bit-by-bit decoding.

Time–space tradeoff

The FSM-based approach reduces decoding overhead by replacing repeated tree traversal with constant-time table lookups, resulting in faster practical performance. While the overall time complexity remains $O(N)$, the number of decoding steps is reduced to approximately $O(\frac{N}{K})$.

However, this comes at the cost of increased memory usage. The FSM requires a lookup table of size $O(2^K \cdot S)$, where S is the number of states, corresponding to the total number of internal

nodes in the Huffman tree. A larger K improves decoding speed by processing more bits per step, but increases memory exponentially. Conversely, a smaller K reduces memory usage but provides less speedup.

2.6.4. Compressed Size Comparison

The following are the compressed results using the ASCII frames and configurations (`output/demo/demo_engine/test_config.json`) from this work's GitHub repository.

Method	Size (KB)
Original	135,684
RLE	13,297
Delta	17,943
Huffman ($K = 1$)	36,885
Huffman ($K = 5$)	36,893
Huffman ($K = 10$)	37,251

Table 3: Compressed size comparison across different algorithms and configurations

Analysis.

- *RLE*: Achieves the best compression, reducing the data size by an order of magnitude. This is due to long runs of identical characters across consecutive frames, which RLE encodes very efficiently.
- *Delta encoding*: Performs well since consecutive frames are largely similar. By encoding only the differences, it reduces inter-frame redundancy. However, these differences are often scattered rather than continuous, making it less effective than RLE.
- *Huffman coding*: Performs worse in this case. The character distribution is relatively uniform, limiting the benefit of variable-length encoding. Additionally, it operates at the symbol (character) level and does not exploit redundancy across frames. For the FSM-based optimization, increasing K introduces exponential memory overhead ($O(2^K)$) without improving compression ratio.

3. Constraint Satisfaction

This section maps the project handout constraints and course rubric to concrete implementation evidence in the repository. Narrative report requirements (context, collaboration, challenges, individual contributions, and lessons learned) are developed in Sections 1, 4, 5, and 6; here we focus on *technical* constraint satisfaction with file-level traceability.

3.1. Constraint-to-evidence matrix

Table 4 is the primary traceability artifact: each row states a constraint, how the codebase satisfies it, where to inspect the implementation, and the observable outcome.

Table 4: Mapping of project constraints to implementation evidence.

Constraint	Approach	Evidence (representative paths)	Outcome
File input/output (text + binary)	Read JSON, WAV, BMP; write plain render log (text), compressed container (mixed text header + binary payloads), optional process log.	src/parsers/json/json.c, src/parsers/wav.c, src/parsers/bmp.c; src/components/render.c, src/components/render_compress.c; src/components/engine.c; src/components/preview.c.	Deterministic artifacts: render text, compressed output, log.
Custom robust parser(s)	Dedicated parsers with typed errors; config layer whitelists keys and validates ranges; engine rejects unknown compression strings.	src/parsers/json/config.c (allowed keys, required fields, defaults); src/parsers/json/json.c; src/parsers/wav.c; src/parsers/bmp.c; engine_prepare_compression in src/components/engine.c.	Invalid files or config fail early with explicit error codes instead of silent corruption.
Clean C + strict warnings	Project build uses strict GCC flags for all translation units and tests.	Makefile (CFLAGS = -Wall -Werror -ansi -pedantic -Iinclude).	Warning-free, standards-oriented compilation baseline.
State machines	Engine runtime is a finite state machine over load, prepare, compress-prep, output, frame loop, and summary stages; Huffman decoding uses an explicit FSM table; preview parses header and frame records sequentially.	engine_run and EngineState in src/components/engine.c; huffman_build_fsm / FSM types in src/compressions/algorithms/huffman.c and include/compressions/algorithms/huffman.h; parse_header, parse_fsm, read_frame in src/components/preview.c.	Clear control flow, stage isolation, and decode structure suitable for complex I/O.
Parser complexity	Multiple non-trivial parsers: JSON subset + config semantics; RIFF/WAVE PCM; BMP DIB/pixel layout; compressed-file header and per-frame records in preview.	As above, plus include/parsers/json/json.h, include/parsers/wav.h, include/parsers/bmp.h, include/components/render_compress.h.	Demonstrates layered parsing beyond a single trivial format.
No external libraries	Implementation uses ISO C standard library; preview uses conditional POSIX vs. Windows calls for timing and screen clear, not third-party frameworks.	Makefile link lines; #ifdef _WIN32 branches in src/components/preview.c.	Self-contained build without dependency managers or non-standard SDKs.
Code quality + feature depth	Modular boundaries (parsers, components, compressions, tests); multiple compression modes (none, rle, delta, huffman) integrated through compress_frame / decompress_frame.	src/compressions/compress.c, src/compressions/decompress.c; src/compressions/algorithms/ (*.c); tests/parsers, tests/components, tests/compressions.	Maintainable structure with substantive algorithmic content.

3.2. File input and output in detail

Inputs. Configuration is a JSON object loaded by `config_load` (`src/parsers/json/config.c`), which requires keys such as `frames_dir`, `fps`, frame range, `palette`, `threshold`, and `compression_algorithm`, with optional `huffman_K` and frame-naming fields. Audio is read as a binary WAV (`src/parsers/wav.c`). Frames are read as BMP files (`src/parsers/bmp.c`) along paths built by `sequence_build_frame_path` (`src/components/sequence.c`).

Outputs. The engine writes a human-readable ASCII timeline via `render_write_frame` (`src/components/render.c`), a compressed stream via `render_write_header` and `render_write_frame_compressed` (`src/components/render_compress.c`), and a textual process log. The `preview` program opens the compressed file in binary mode, parses textual header lines and embedded FSM metadata when applicable, reads binary frame payloads, and prints decoded ASCII (`src/components/preview.c`). This satisfies the handout requirement for file-based processing across both text and binary modalities.

Compressed Output Format.
The compressed file follows a hybrid **text + binary** format. The header is stored as human-readable text, followed by optional FSM metadata (for Huffman), and then per-frame binary payloads.

Header. The file begins with global metadata:

- `FORMAT`: file identifier (e.g., ASCII_VIDEO)
- `WIDTH`: frame width
- `HEIGHT`: frame height
- `COMPRESSION`: compression type identifier (0: None, 1: RLE, 2: Delta, 3: Huffman)
- `K`: number of bits processed per step in the Huffman FSM (only applicable when Huffman compression is used)
- `NUM_STATES`: number of FSM states

FSM Section (Huffman only). If Huffman compression is used, the FSM is serialized between `FSM_BEGIN` and `FSM_END`. Each line represents one transition:

```
state input next_state bits_used output_length [symbols...]
```

where:

- `state`: current FSM state
- `input`: K-bit input (as integer)
- `next_state`: next FSM state
- `bits_used`: number of bits consumed
- `output_length`: number of decoded characters
- `symbols`: ASCII values of output characters, separated by spaces; '-' if `output_length` is 0

Frame Section. Each frame is stored with metadata followed by compressed binary data:

- `FRAME`: frame index
- `TIME`: timestamp

- `DATA_BITS`: number of valid bits in the payload
- `DATA_BEGIN` / `DATA_END`: delimit binary data

The actual compressed data is stored as a packed bit stream, where bits are written into bytes and read using a bit-level reader during decoding.

Design Rationale. This format separates structure from data: textual headers and FSM metadata provide readability, while binary payloads ensure compact storage. The FSM representation allows efficient decoding without reconstructing the Huffman tree, enabling fast multi-bit processing during playback.
These satisfy the handout requirement for file-based processing across both text and binary modalities.

3.3. Parser robustness and incorrect-input detection

Robustness is implemented through *validation at boundaries* and *typed error returns* rather than ad-hoc crashes:

- **JSON and config:** unknown keys are rejected (`config_validate_allowed_keys`); required keys must be present and correctly typed; string length and numeric ranges are checked (`config_validate`). Frame extension is restricted to `.bmp` and `frame_digits` is capped, preventing ambiguous or oversized paths.
- **WAV and BMP:** parsers verify signatures, header consistency, and supported format assumptions appropriate to the course scope (PCM WAV; uncompressed BMP suitable for the frame pipeline).
- **Compression selection:** strings other than `none`, `rle`, `delta`, and `huffman` fail in `engine_prepare_compression`; Huffman additionally requires `huffman_K > 0` (`src/components/engine.c`).

Table 5 summarizes typical fault categories and the defensive response pattern (exact error codes are enumerated in the respective headers under `include/parsers/` and `include/components/`).

Table 5: High-level parser fault categories and system response.

Fault category	Example	Response pattern
Structural / syntax	Invalid JSON; malformed WAV/BMP headers	Parse fails; `config_load` or low-level parser returns non-success; engine aborts in error state.
Schema / policy	Unknown config key; wrong value type; invalid frame range	`CONFIG_ERR_UNKNOWN_KEY`, `CONFIG_ERR_WRONG_TYPE`, or range errors; safe re-init of config.
Unsupported media	Non-conforming BMP or WAV for implemented subset	Load error at BMP/WAV layer; engine surfaces failure without emitting partial outputs.
Invalid compression mode	Unrecognized `compression_algorithm` string	`ENGINE_ERR_CONFIG` from `engine_prepare_compression`.

Automated tests under `tests/parsers`, `tests/components`, and `tests/compressions` exercise many negative paths; full end-to-end engine tests are lighter and remain a documented improvement opportunity (Section 6).

3.4. Constraint pipeline (diagram)

Table 4 is intentionally dense; Figure 8 gives the same end-to-end structure in visual form: heterogeneous inputs, parser layer, engine and rendering, compressed output, and preview playback.

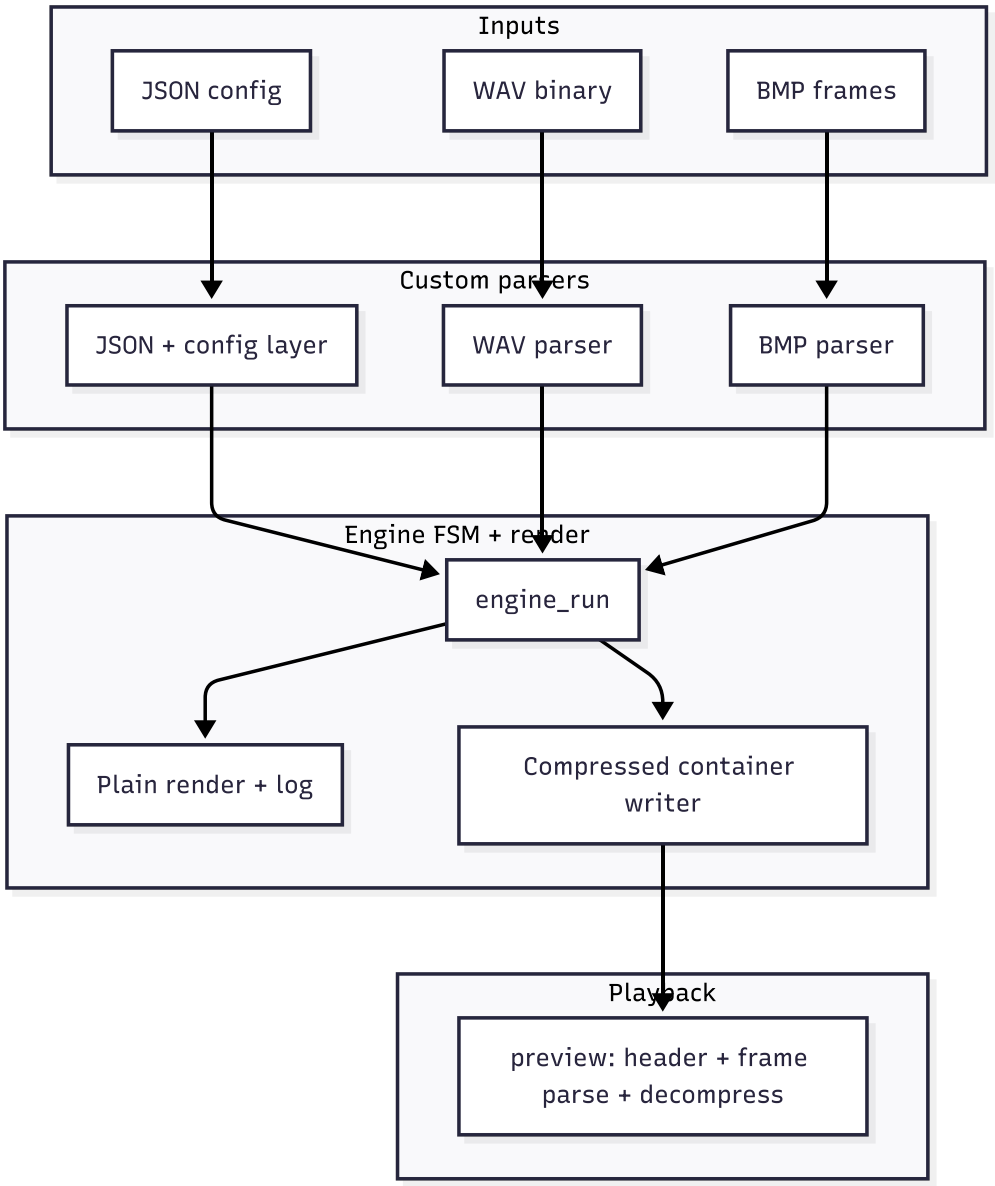


Figure 8: High-level data flow and rubric-oriented module boundaries: inputs through parsers and engine to plain and compressed outputs, then preview. Complements the constraint-to-evidence matrix in § 3.1.

4. Development Process and Team Collaboration

This section explains how a two-person team organized design, implementation, and validation, and how version control and shared build conventions kept the codebase consistent across machines.

4.1. Development timeline and phases

Work proceeded in four overlapping phases. The boundaries were not strictly sequential—for example, tests were added alongside features—but the phases help explain how risk was managed.

Table 6: High-level development phases and concrete artifacts.

Phase	Focus	Representative artifacts
Design	Agree on module boundaries, configuration schema, compressed-container layout, and error-handling style (typed return codes).	Headers under `include/`; early stubs in `src/parsers/`, `src/components/`, `src/compressions/`.
Parallel implementation	Implement parsers, engine orchestration, ASCII rendering, compression algorithms, and preview consumption along separate but stable interfaces.	`src/parsers/json/config.c`, `src/components/engine.c`, `src/compressions/`, `src/components/preview.c`.
Integration	Connect engine output to compression and logging; align config keys (e.g. `compression_algorithm`, optional `huffman_K`) with runtime behavior.	`src/components/render_compress.c`, `output/demo/demo_engine/`.
Validation	Run unit tests per module; exercise end-to-end runs with demo configs and inspect `output/` logs and renders.	`tests/`, `Makefile` targets `test` / `run_test`.

4.2. Work split for a two-person team

The project naturally splits into a **media ingestion and orchestration track** and a **compression and delivery track**. Both tracks are required for the product narrative (correct rendering *and* usable compressed output with preview).

- **Subsystem A — parsers, engine, and raw rendering.** One teammate focused on JSON configuration loading and validation, WAV and BMP parsing, frame sequencing, the engine state machine, ASCII conversion, plain-text render output, and process logging. This track establishes the functional contract: what a "frame" means, how audio lines up with frames, and what the uncompressed render records look like.
- **Subsystem B — compression, decompression, and preview.** The other teammate focused on algorithm implementations (including symbol statistics and Huffman table/FSM preparation where applicable), the compressed file format, `compress_frame`/`decompress_frame` integration, and the terminal preview path that reads headers and frame payloads. This track delivers storage-efficient artifacts and user-facing playback.

Shared ownership. Interface decisions (e.g. fields in `include/parsers/json/config.h`, structures in `include/components/render_compress.h`) and naming conventions were agreed explicitly so that changes on one side did not silently break the other. End-to-end behavior was validated together using shared demo inputs under `output/demo/`.

4.3. Version control with GitHub

We used **Git** with a **GitHub** remote as the single source of truth. This choice supported parallel development with minimal merge pain:

- **Isolated changes:** Work on parsers/engine and on compression/preview touched mostly disjoint file trees (`src/parsers/`, `src/components/engine.c` *vs.* `src/compressions/`, `src/components/preview.c`), which reduced the chance of conflicting edits in the same region of a file.
- **Small, reviewable steps:** Features were integrated through the shared remote using direct pushes to a main **integration branch** (exact branching policy is a team convention; the important point is that history remained centralized and inspectable).
- **Traceability:** Commit messages and pull requests (when used) document *what* changed and *why*, which complements the process log produced at runtime by the engine.

GitHub did not replace local validation: every integration still had to compile and pass targeted tests on each machine (§4.5).

4.4. Integration strategy and interface contracts

Integration risk was concentrated at a few well-defined boundaries:

1) **Configuration → engine.** The JSON schema enforced by `config_load` (`src/parsers/json/config.c`) determines frame paths, timing parameters, palette, threshold, and compression mode. Invalid or unknown keys fail early, which avoids partial runs with ambiguous state.
2) **ASCII frame → compression.** The engine produces structured ASCII output and passes frame payloads into the compression layer through the shared types declared in `include/components/render_compress.h` and `include/compressions/compress.h`. Agreement on width/height, compression enum, and Huffman parameters (`huffman_K`) was essential for round-trip correctness.
3) **Compressed file → preview.** The preview program (`src/components/preview.c`) must parse the same textual header markers and binary frame layout that the writer emits. Any change to the on-disk format required coordinated updates on both sides.

When integration issues appeared (e.g. mismatched frame naming or compression strings), we used **reproducible demo configs** and **diffs in Git** to locate whether the fault lay in parsing, engine logic, or the codec path.

4.5. Reproducibility across team machines

Build system. The project is built with `make` from a root `Makefile`. Compiler flags (`-Wall -Werror -ansi -pedantic -Iinclude`) apply uniformly, so a warning on one laptop is a warning for everyone. Targets such as `retrovision` and `preview` encode the link line and object lists, reducing "works on my machine" drift.
Tests. Unit tests live under `tests/` and are compiled on demand, e.g. `make run_test TEST=tests/parsers/test_wav.c`. Sharing a small set of *known-good* test invocations in chat meant both members verified the same behavior before merging.
Demo artifacts. The `output/demo/` tree (including `output/demo/demo_engine/`) provides sample configs, logs, and outputs for cross-checking integration. Comparing `process.log` and compressed outputs after a merge quickly surfaces regressions.
Environment assumptions. Both sides used a standard C toolchain (`gcc`) and POSIX-style shells; preview code accounts for Windows vs. POSIX timing (`src/components/preview.c`). No external dependency manager was required, which simplified onboarding.

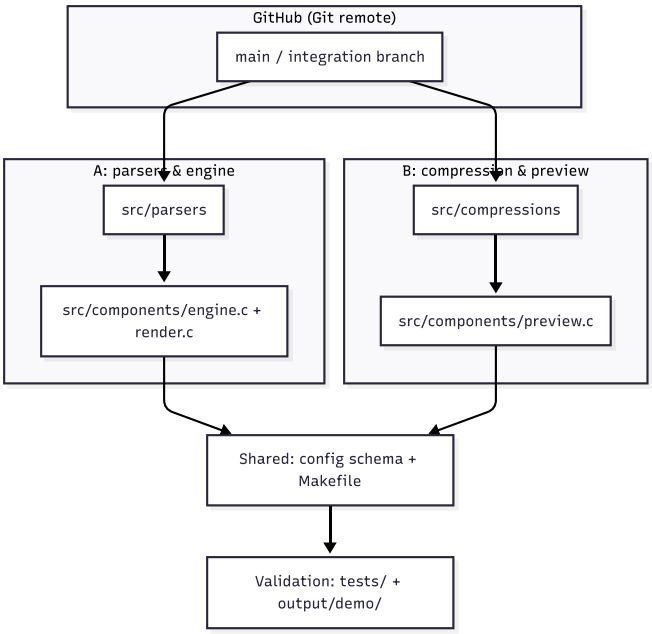


Figure 9: Parallel workstreams converging on shared contracts and a Makefile-driven build.

5. Challenges, Trade-offs, and Engineering Decisions

5.1. Heterogeneous inputs and parser robustness

The system must accept three *different* input modalities: a text-based JSON configuration, a structured binary WAV stream, and bitmap frames with a binary DIB layout. Each format has its own failure modes (invalid signatures, inconsistent header fields, unsupported bit depths or audio encodings, malformed JSON).

Challenge. A single "generic" read loop cannot validate all three; each parser needs domain checks, yet the engine must see a uniform success/failure vocabulary.

Response. We implemented dedicated parsers with small, explicit APIs and typed error enums (e.g. `BmpError`, `WavError`, `ConfigError`), then mapped failures into engine-level outcomes. Configuration adds a **whitelist of keys** and type checking so corrupt policy cannot silently propagate into runtime.

Trade-off (strictness *vs.* flexibility). Strict rules—such as rejecting unknown JSON keys and requiring `.bmp` frame extensions in configuration validation—improve safety and grading clarity, but reduce "bring any file" flexibility. A more permissive design would need richer diagnostics and broader format support, which was out of scope for a course-sized engine.

5.2. Temporal synchronization and audio-driven highlighting

Video frames are discrete, while audio is a continuous waveform sampled at a rate that need not be an integer multiple of the configured `fps`. The engine maps each frame index to a half-open sample interval using **integer division** in C:

```
start_sample = (relative_index * sample_rate) / fps
end_sample = ((relative_index + 1U) * sample_rate) / fps
```

(See `src/components/engine.c` in `engine_process_one_frame`, before `wav_average_amplitude`.)

Challenge. Rounding partitions the audio timeline into windows that are approximately one frame long but not perfectly equal in samples; extreme `fps`/sample-rate combinations can make boundary effects visible in thresholding behavior.

Response. We tried to keep the mapping **deterministic and documented**: average absolute amplitude over each window is compared to a single user threshold to set a boolean highlight flag. This is simple to explain, test, and reproduce across machines.

Trade-off (accuracy *vs.* simplicity). More advanced alignment (sub-frame interpolation, perceptual loudness, or peak detection) could track human hearing better, but would add complexity and subjective tuning beyond the project brief.

5.3. Raw fidelity, storage footprint, and compression modes

Uncompressed ASCII timelines are excellent for debugging and inspection, but can be very large for long frame ranges. The compression track targets smaller artifacts and preview playback.

Challenge. Different algorithms optimize for different statistics: run-length and delta coding are cheap conceptually but vary with frame content; Huffman coding can exploit global symbol frequencies but requires additional preparation.

Response. The engine supports multiple modes (`none`, `rle`, `delta`, `huffman`) selected via configuration. For Huffman, the implementation performs a **pre-pass** over the frame range (`engine_collect_stats` in `src/components/engine.c`) to accumulate byte-level symbol counts before building codes and decoder tables. That guarantees a consistent codebook for the sequence at the cost of extra work up front.

Trade-off (compression ratio *vs.* encode/decode complexity). Stronger compression often implies more book-keeping (tables, FSM metadata in the compressed file, preview reconstruction). We accepted that complexity because it directly supports the product goal of *usable* outputs, not only raw dumps.

Table 7: Representative engineering trade-offs and adopted resolutions.

Decision axis	Benefit	Cost / risk	Chosen direction
Parser strictness	Predictable failures; easier grading and debugging.	Rejects some "almost valid" media or configs users might try.	Strict config + format subset (JSON/WAV/BMP) with clear errors.
Audio windowing	Simple, reproducible frame–audio mapping.	Integer division yields slightly uneven windows; threshold tuning sensitive.	Fixed formula + threshold flag in JSON config.
Raw + compressed outputs	Transparency (raw) and practical storage/playback (compressed).	Two output paths to maintain; preview must track writer format.	Emit both; document container layout in code (`render_compress.h`).
Huffman pre-pass	Global frequencies improve coding for the whole clip.	Extra full-frame traversal before final encode; more implementation work.	`COLLECT_STAT` stage in engine FSM; used only when needed.
Engine error buckets	Stable top-level outcomes for the CLI.	Fine-grained BMP/WAV causes collapse to a few `ENGINE_ERR_*` codes.	Keep module-level enums for tests; engine returns coarse codes.

5.4. State-machine orchestration and maintainability

The engine coordinates many steps (load config, load audio, prepare sequence, optional statistics pass, compression setup, open outputs, per-frame loop, summary). A monolithic "mega-function" would be difficult to reason about under `-Wall -Werror -ansi -pedantic`.

Response. A finite state machine in `engine_run` (`src/components/engine.c`) isolates stages and makes failure handling explicit: a stage error transitions to a terminal error state and returns a single engine code.

Trade-off. More states mean more bookkeeping when adding features, but the structure paid off whenever compression logic or output paths evolved, because new work could be scoped to one or two states.

5.5. Risk handling, testing, and debugging

Module tests under `tests/parsers`, `tests/components`, and `tests/compressions` focus on boundary conditions and negative inputs (malformed files, missing keys, wrong types). They do not replace a full end-to-end harness for every engine failure mode, but they **anchor** parser and codec behavior so refactors are safer.

Operational debugging. The textual `process.log` and reproducible command lines (`make`, `retrovision`, `preview`) allowed us to compare behavior before and after a merge—complementing Git history with runtime evidence.

6. Outcomes, Learning, and Future Work

This closing section states what the team delivered, why the compression and preview path matters for users, what we learned from building the system in C with minimal dependencies, and which improvements would come next. **Version control on GitHub** supported low-friction integration (Section 4); the outcomes below are about *product and technical* results rather than merge mechanics.

6.1. Project outcomes

The completed system is an end-to-end **ASCII video engine** with a **playback-oriented delivery layer**:

- **Inputs:** JSON configuration (`compression_algorithm`, frame range, palette, threshold, optional Huffman parameters, frame naming rules), a PCM WAV file, and a sequence of BMP frames (`src/parsers/json/`, `src/parsers/wav.c`, `src/parsers/bmp.c`).
- **Core processing:** A state-machine-driven engine loads each frame, maps it to an audio sample window, derives a highlight flag from average amplitude, renders ASCII, and writes a human-readable timeline plus a process log (`src/components/engine.c`, `src/components/render.c`).
- **Compression and packaging:** Frame payloads are compressed according to the selected mode and written with a documented header and per-frame records (`src/compressions/`, `src/components/render_compress.c`).
- **Playback:** The `preview` program reads the compressed file, reconstructs context (including Huffman-related metadata when applicable), decompresses each frame, and displays animated ASCII in the terminal (`src/components/preview.c`).
- **Build and validation:** A single `Makefile` builds `retrovision` and `preview` with strict flags (`-Wall -Werror -ansi -pedantic`); targeted tests under `tests/` support regression checking after changes.

6.2. User experience: compression and preview are core

The compressed output and preview loop are not an optional add-on. They address a practical constraint that appears immediately in real use: **raw ASCII render files grow quickly** with frame count and resolution, which makes storage, transfer, and repeated inspection costly.

Why this matters. A user (or grader) can still open the plain render for line-by-line verification, but day-to-day interaction—checking motion, timing, and highlight behavior—is far more feasible against a compressed artifact plus `preview`. We therefore treats compression/decompression and preview as **first-class product requirements**, aligned with the course emphasis on file processing and stateful decoding, not only on rendering correctness.

6.3. What we learned

Parser and engine track.
We deepened experience with **binary file discipline** (WAV/BMP layout assumptions, careful reads, and early validation), **JSON-as-config** with strict schema policy, and **state-machine orchestration** for multi-stage pipelines. We also learned how quickly "small" integration details (e.g. frame path templates) become user-visible failures when config and data disagree.

Compression and preview track.
We learned to treat codecs as **contracts**: the encoder and decoder must agree on symbols, bit packing, and container markers. Implementing Huffman preparation (e.g. symbol statistics before the final pass) reinforced that compression is not only algorithmic cleverness but also **systems engineering** (memory, determinism, and on-disk interoperability with `preview`).

6.4. Future improvements

The following items are **planned** enhancements; they are not required to claim the current milestone, but they are the natural next engineering steps:

1) **End-to-end integration tests:** A small harness that runs `retrovision` on a tiny fixture (few frames + short WAV) and asserts non-empty outputs, stable exit codes, and round-trip preview behavior would guard regressions across parser, engine, and compression boundaries.

2) **Benchmark automation:** Scripted runs to record wall time and output sizes for each compression mode (numbers should be produced by scripts, not invented in prose).

3) **Broader format support and diagnostics:** Optional extensions might include additional WAV encodings or BMP variants, plus richer error messages (while keeping the "no mystery dependencies" philosophy).

4) **Documentation of expected artifacts:** A short maintainer note listing canonical demo commands and expected files under `output/demo/` would further lower onboarding cost for new contributors.

6.5. Closing remarks

The project demonstrates that a small pure-C codebase can still deliver a credible multimedia-to-text pipeline: custom parsers, explicit state machines, mixed text/binary I/O, and a user-facing compression and preview story. The hardest lessons were not about any single function, but about **keeping contracts stable** across modules while preserving strictness and reproducibility. That balance—technical depth with pragmatic usability—is what we hope a reader takes away as the project's primary outcome.